YuMi Learning to Grasp
FRTN01
Real-Time Systems

Markus Lantto - markus.a.lantto@gmail.com
Karl Sköld - kalleskold01@gmail.com
Jonathan Stoltz - j.stoltz02@gmail.com
Linnéa Westlin - li2330we-s@student.lu.se

Supervisor - Ahmed Albayati

May 2026

1 Introduction

For the *YuMi Learning to Grasp* project, the central problem to be solved was writing a program where YuMi (a dual arm robot from ABB) moves a tray from one location to another using its arms. YuMi is mounted on an omni-directional moving platform which will be referred to as "Sleipner" throughout the report. Sleipner is partially developed by the Control Department at LTH. The dimensions of the tray, and its rough location, meaning its coordinates relating to the origin of YuMi, are given knowledge at the start of the project.

To determine a more precise position of the tray, YuMi's sensors will have to be utilized. More specifically, YuMi's pressure sensors located on its wrists will be used to determine an "exact" location of the tray. The pressure sensors will be used in a way that if YuMi detects contact with any object somewhere around the rough location of the tray, it will be assumed that this is an edge of the tray. Then, using simple geometry calculations, the exact location of the tray can be determined.

YuMi should move the grippers to the correct position. The tray will then be picked up. Following this, Sleipner will move 1 meter backwards from the table, move 1 meter forward again and then place the tray on the table again.

The process is controlled through a computer that YuMi and Sleipner are connected to. The computer runs python code on a Linux system that works on top of ROS (Robot Operating System) which communicates with YuMi.

2 Program Structure

The system is implemented as a ROS Noetic node in Python, structured around a sequentially executed pipeline. The sequence is shown in Figure 1 below. It is directed by a main control thread and callback threads.

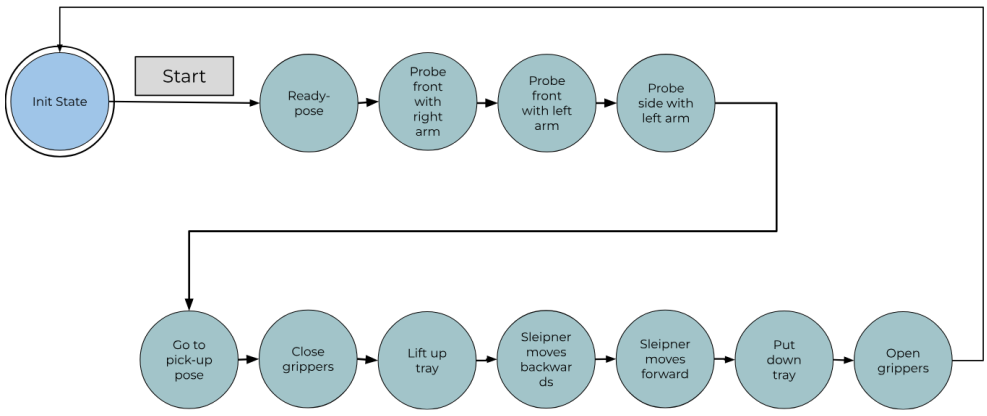


Figure 1: Execution sequence for the robot program.

Because Python and ROS 1 are used for the project, it is impossible to obtain a true deterministic real-time performance (as a consequence of Python not being a true real-time language). However, it is still possible to fulfill "soft" real-time requirements, such as breaking a while-loop when the condition of the loop changes when a ROS topic delivers new messages or values. This is accomplished by sampling sensor data concurrently while executing the joint probing motions, using conditional while loops that monitor state changes. The code architecture below should be read with the code appendix as a foundation. The code is structured with the following components:

Main Control Thread: This thread operates the central pipeline for the YuMi robot. It initializes the ROS node, manages the publishers and subscribers, and contains the logical operational flow of the program:
Ready Pose ⇒ Probing Front ⇒ Probing Side ⇒ Geometry Calculation ⇒ Pick and Lift ⇒ (wait for Sleipner motion) ⇒ Place tray ⇒ Retract hands.

It handles the decision making by evaluating sensor data and thresholds inside execution loops, and publishes motion trajectories to the trajectory topic, which in turn are translated and sent to YuMi's hardware via EGM (Externally Guided Motion).

Pressure Sensor Callback Thread: A callback thread that subscribes to the force and pressure data that are published by the pressure sensors in YuMi's hands. It registers all forces, but whenever a physical collision over 0.2N is detected (which was decided by experimental testing to be the threshold signaling that the grippers have touched the tray), the main thread will register it as a collision with the tray.

Position Callback Thread: A callback thread that actively listens to the real-time positional data of the grippers that are published by the coordinated joint states in YuMi's hands.

Geometric grip position calculator: A help function called by the main thread immediately after the probing phase. It takes the inputs of the raw global coordinates (P_1, P_2, P_3) saved during the probing states and applies a geometric calculation to extract the position of the tray in relation to YuMi's origo. Then it computes the desired gripping coordinates for YuMi's grippers. For this program, the gripping coordinates were decided to be on the long side of the tray, 10 cm from the edges of the front-facing edge (towards YuMi) of the tray. The exact equations are formulated as follows:

First, the orientation angle θ of the tray's front edge relative to YuMi's base coordinate system is computed using the misalignment between the two front probing points (P_1, P_2):

$$\theta = \text{atan2}(y_2 - y_1, x_2 - x_1) \tag{1}$$

To locate the coordinates for the left corner of the tray (which is referred to as the local origin of the tray, x_O, y_O), the vector from P_1 to the side probing point P_3 is projected onto the front edge line. The scalar projection distance d along this edge is defined as:

$$d = (x_3 - x_1)\cos\theta + (y_3 - y_1)\sin\theta \tag{2}$$

The coordinates of the left corner origin are then calculated by moving from P_1 along

the front edge:

$$x_O = x_1 + d\cos\theta, \quad y_O = y_1 + d\sin\theta \tag{3}$$

Finally, the help function derives the desired grasping positions for the left (G_L) and right (G_R) grippers based on the width of the tray $W_T = 0.45$ m and the 10 cm inward from the corners along the edge of the tray, as mentioned above. Using these geometric equations, the final gripping coordinates are calculated as follows:

$$x_{GL} = x_O - 0.1\cos\theta, \quad y_{GL} = y_O - 0.1\sin\theta \tag{4}$$

$$x_{GR} = x_O - (W_T - 0.1)\cos\theta, \quad y_{GR} = y_O - (W_T - 0.1)\sin\theta \tag{5}$$

3 Operator Communication

The communications to YuMi can be made more or less complex depending on how much control is desired over the exact position of the different joints and how they reach the desired state. Since the project does not require very complex control of the different joints (meaning it typically isn't of importance *how* the arms reach a certain position) the simplest method will be used, where the arguments to be sent to YuMi itself will be the coordinates and orientations of the arms, desired width between the two gripper fingers on each arm, and the time YuMi is given to reach this desired state. For Sleipner, the needed arguments will simply be its desired velocity and the time to go from the current velocity to the desired velocity (essentially acceleration).

4 Control Principles

Control principles are not very widely applicable in this project, since most controllers used are already pre-defined. All joints in YuMi's arms have built-in controllers that take a position and a time to reach that position and generate joint velocities that follow the desired trajectory. The same thing can be said about Sleipner, which also is a fully defined system with built-in low-level controllers. Therefore, there is no need to create any PID controllers for its wheels, it is enough to send a navigation command and Sleipner will automatically handle the physical execution.

5 User Interface

In order to control YuMi, Sleipner and run programs, a PC connected to the robot over Ethernet is used. Several terminals running in parallel are needed. Firstly, it has to be decided if the program should run in the simulation environment, RViz, or on the real robots. All terminals are then started and initialized with the command "USE_REAL" or "USE_SIM". Depending on which command is used, the environment variables required to set up communication with either the physical hardware or the simulation environment are then determined.

The terminals used for each functionality when running the real (or simulated) process in the order they should be opened are listed in Table 1 below. The terminals should be started in the order that they are mentioned.

Terminal	**Function**	**Required in Simulation**
1	Starts the robot (Roslaunch)	Yes
2	Sets up the communication link EGM	No
3	Starts the trajectory controllers for YuMi	Yes
4	Starts the pressure sensors	No
5	Calibrates the pressure sensors	No
6	Starts the trajectory controllers for Sleipner	No
7	Run the desired program	Yes

Table 1: Description of the functionality in each terminal

The commands for starting the right processes (using the real process, not simulation) in the 7 mentioned terminals are:

1. *roslaunch yumift_common bringup.launch use_real:=true*
2. *roslaunch yumift_common start_egm.launch __ns:=yumi*
3. *rosrun yumift_controllers trajectory_controllers.py*
4. *roslaunch yumift_common sensors.launch*
5. *rosrun yumift_controllers ftsensor_calibration.py*
6. *rosrun sleipner_controllers trajectory_controller.py*
7. –

The Roslaunch command starts up several nodes on the robots, while EGM (Externally Guided Motion) allows the PC to communicate with and control the process.

The trajectory controllers for YuMi and Sleipner are used by our program to move the arms and the omnidirectional platform using cartesian coordinates of desired position or incremental movement. The pressure sensors are vital in order to find the location of the tray. When running only the simulation, the EGM, the pressure sensors and Sleipner controllers become unnecessary. For testing the program in RViz, the incremental probing until the pressure sensors detects the tray were replaced with a fixed number of incremental steps, using for-loops. Figure 2 shows the simulation environment in RViz.

Once the program is started, it runs automatically and does not require any additional inputs by the user. The user will receive information via prints to the terminal on several important occasions. This include different trajectories sent to YuMi and Sleipner, the coordinates of the edges of the tray located by the grippers and the coordinates where the tray should be picked up. Some specific examples of prints when locating the tray include "Probing position 1 with right arm", followed by "Current x force is: xx" and "Saved right arm pos1 is: x y" when the tray is located.

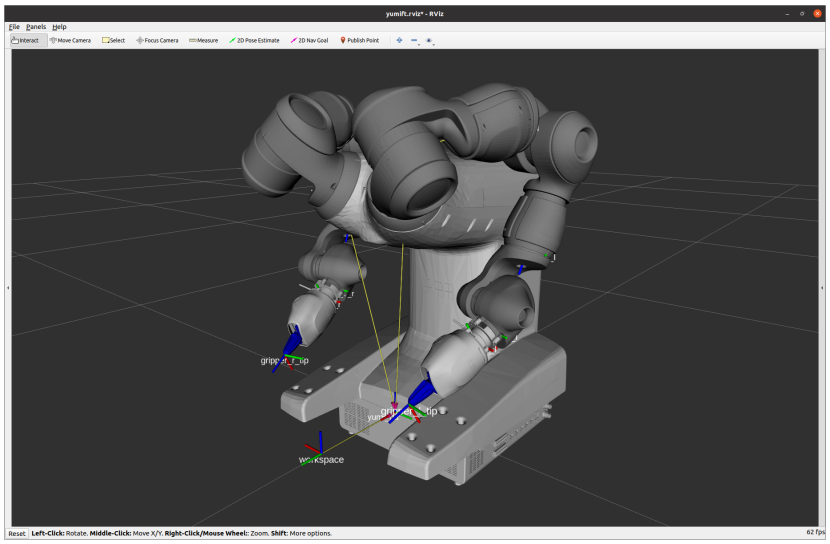


Figure 2: Simulation in RViz

6 Results

The results of the project aren't necessarily quantifiable, it is mostly based on either meeting the goal, or not. However, the results were highly satisfactory. The tray was able to be identified, moved, and brought back to the same position with very high repeatability success. The results from an iteration where the tray was not perfectly perpendicular to the robot are presented in Figure 3.

Problems did arise in some edge cases where the tray was either too close, or too far away, from YuMi before the program was started. In the case of the tray being too close to YuMi, this caused problems due to the fact that one of the arms wasn't fully functional, and therefore certain configurations of the arm that were too close to the body would cause the stress to be too high so YuMi would shut down. The case where the tray was too far away would cause problems because Sleipner, due to safety concerns, can only go within a certain distance of the table that the tray is standing on. This would then make it impossible to locate and grab the tray without reaching a singularity, due to the arms being too stretched out if the tray is located too far away.

In figure 4, a sped up video of the whole process can be seen.

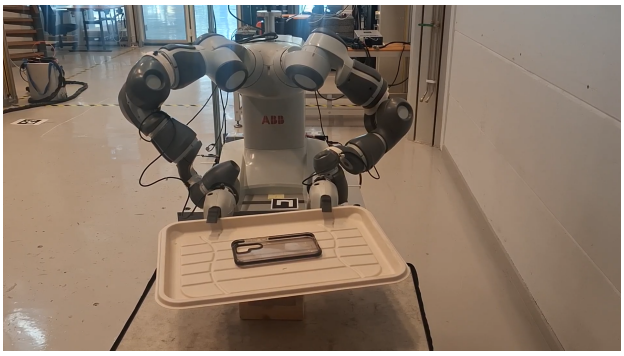


Figure 3: Gripping points located when the tray wasn't completely perpendicular.

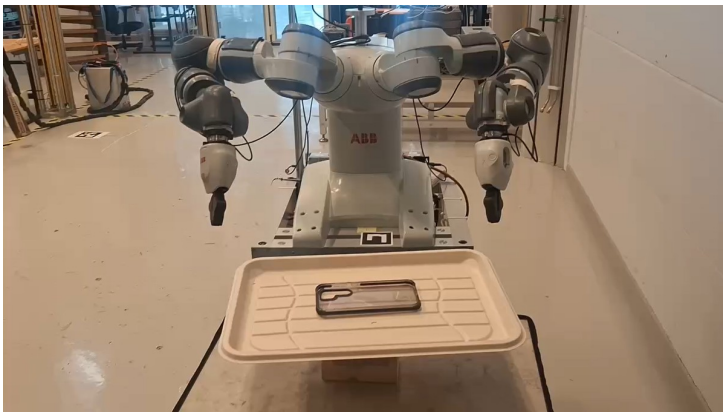


Figure 4: Click the image to open the video of YuMi performing the routine.

7 Conclusion

The project successfully demonstrates that YuMi can identify, grasp, and relocate a tray with high precision. However, there are several ways the process could improve. A significant challenge was the occurrence of singularities, which occasionally prevented the robotic arms from reaching certain positions. Currently, the program gives the arms a desired position and orientation without defining an optimized or safe path to get there. This could possibly be solved by implementing more extensive protection against singularities with motion planning logic. Another possible feature could be a virtual bounding box restricting the arms from trying to move out of their physical reach.

While the current probing method is fully functional, it suffers from a small delay due to only checking the measured pressure after every incremental movement. This sometimes causes a small displacement of the tray before the probing routine is terminated. This could be improved by changing to an interrupt-driven architecture or using a high-priority thread for detecting pressure.

Moreover, some problems with the hardware were detected, making the performance less reliable. The right gripper has different dimensions and lower stability while grasping and transporting the tray compared to the left gripper, as seen in Figure 2. This limits the possible load on the tray while keeping it stable during transportation. The left arm sometimes lowers significantly before getting to the position for picking up the tray, limiting the initial position of the tray compared to the initial position of the robot.

In conclusion, the project has been successful, as YuMi and Sleipner can complete the desired task with high repeatability. The results demonstrate the effectiveness of using ROS to control a complex robotic system for multi-stage tasks. Addressing the suggested improvements would further enhance the system's robustness while increasing flexibility regarding tray placement and load capacity.

8 Appendix

Listing 1: Project code.

```
#!/usr/bin/env python3
import rospy
import math

from geometry_msgs.msg import WrenchStamped, Quaternion
from yumift_msgs.msg import RobotState
from yumift_msgs.helper import Helper as H

current_force_x_right = 0.0
current_force_x_left = 0.0
current_pos_x_right = 0.0
current_pos_y_right = 0.0
current_pos_x_left = 0.0
current_pos_y_left = 0.0

#Helper method to move Sleipner (incremental movement from local coordinates)
def move_sleipner(x : float, y : float, z : float, t : float, pub):
    msg = Quaternion()
    msg.x = x
    msg.y = y
    msg.z = z #rotation
    msg.w = t #time

    pub.publish(msg)
    print("Moving Sleipner:", x, y)

#Calculating the gripping points from the probing points and the geometry of the tra
def calc_geometry(p1_x, p2_x, p3_x, p1_y, p2_y, p3_y):
```

```
    tray_width = 0.45  # Tray Width
    grip_depth = 0.1   # 10 cm in from the corners

    # Calculating the angle offset (90 degrees = Completely straigth)
    dx = p2_x - p1_x
    dy = p2_y - p1_y
    theta = math.atan2(dy, dx)

    # Distance d
    d = (p3_x - p1_x) * math.cos(theta) + (p3_y - p1_y) * math.sin(theta)

    # Calculating left corner origo
    origo_x = p1_x + d * math.cos(theta)
    origo_y = p1_y + d * math.sin(theta)

    # Coordinates for new origo

    # Left grip
    gl_x = origo_x - grip_depth * math.cos(theta)
    gl_y = origo_y - grip_depth * math.sin(theta)

    # Right grip
    gr_x = origo_x - (tray_width-0.1) * math.cos(theta)
    gr_y = origo_y - (tray_width-0.1) * math.sin(theta)

    # Printing values
    print(f"Angle: {math.degrees(theta):.2f} degrees")
    print(f"Origo: X={origo_x:.3f}, Y={origo_y:.3f}")
    print(f"Left grip: X={gl_x:.3f}, Y={gl_y:.3f}")
    print(f"Right grip: X={gr_x:.3f}, Y={gr_y:.3f}")
    return [gl_x, gl_y, gr_x, gr_y]

#Gets the current force from the right wrist in x-direction
def force_callback_right(msg):
    global current_force_x_right
    current_force_x_right = abs(msg.wrench.force.x)

#Gets the current force from the left wrist in x-direction
def force_callback_left(msg):
    global current_force_x_left
    current_force_x_left = abs(msg.wrench.force.x)

#Gets the current position of the grippers
def pos_callback(msg):
    global current_pos_x_right
    global current_pos_y_right
    global current_pos_x_left
```

```
    global current_pos_y_left

    current_pos_x_right = (msg.pose[0].position.x)
    current_pos_y_right = (msg.pose[0].position.y)
    current_pos_x_left = (msg.pose[1].position.x)
    current_pos_y_left = (msg.pose[1].position.y)

def main():
    # First, we need to add ROS node capabilities to this Python script.
    # This essentially connects the script to the "ROS network".
    rospy.init_node("example_1_simple_trajectory", anonymous=True)
    rospy.Subscriber("/yumi/sensors/wrench/right/compensated_sensor", WrenchStamped
    rospy.Subscriber("/yumi/sensors/wrench/left/compensated_sensor", WrenchStamped,
    rospy.Subscriber("/yumi/unified/robot_state_coordinated", RobotState, pos_callba
    sleipner_pub = rospy.Publisher("/base/target", Quaternion, queue_size=1)

    # Go to ready pose
    H.quick_send(
        topic="/trajectory",
        routine_name="ready_pose",
        print_message="READY_POSE sent")

    #Posture before first probing, grippers facing down with their flat sides forwa
    INIT_posture = H.posture(
        # time to get to this posture, in [s]
        5.0,
        (H.cm(35, -10, 17.3), H.e2q(180, 0, -90), 0.0),
        (H.cm(35, 10, 18), H.e2q(180, 0, +90), 0.0)
    )

    #Incremental movement forward with right gripper to locate the tray
    PROBE_posture_r = H.posture(
        0.25,
        primary=(H.cm(0.25, 0, 0), H.e2q(0, 0, 0), 0.0),
        secondary=tuple(),
        incremental=H.Incr.GLOBAL
    )

    #Incremental movement forward with left gripper to locate the tray
    PROBE_posture_l = H.posture(
        0.25,
        primary=tuple(),
        secondary=(H.cm(0.25, 0, 0), H.e2q(0, 0, 0), 0.0),
        incremental=H.Incr.GLOBAL
    )

    #Incremental movement right from the side with left gripper to locate the tray
```

```
PROBE_posture_side_inc = H.posture(
    0.5,
    primary=tuple(),
    secondary=(H.cm(0, -0.5, 0), H.e2q(0, 0, 0), 0.0),
    incremental=H.Incr.GLOBAL
)

#Moves the left arm to the left of the tray
PROBE_posture_side = H.posture(
    5,
    primary=tuple(),
    secondary=(H.cm(0, 20, 0), H.e2q(0, 0, 90), 0.0),
    incremental=H.Incr.GLOBAL
)

#Moves YuMi to INIT_posture
H.quick_send(
    topic="/trajectory",
    print_message="Going to INIT posture",
    wait_completion=True,  #Rospy sleeps until the trajectory is finished
    postures=[
        INIT_posture,
])

rospy.sleep(5.0)

#The right arm moves incrementally forward until the tray triggers the force sen
while(current_force_x_right<0.2):
    H.quick_send(
        topic="/trajectory",
        print_message="Probing position 1 with right arm",
        wait_completion=True,
        postures=[
            PROBE_posture_r,
])
print("Current x force is:", current_force_x_right)
print("Current x pos is:", current_pos_x_right)
print("Current y pos is:", current_pos_y_right)

# Saving pos1
pos1 = [current_pos_x_right,current_pos_y_right]

#The left arm moves incrementally forward until the tray triggers the force sens
while(current_force_x_left<0.2):
    H.quick_send(
        topic="/trajectory",
        print_message="Probing position 2 with left arm",
```

```
        wait_completion=True,
        postures=[
            PROBE_posture_l,
])
print("Current x force is:", current_force_x_left)
print("Current x pos is:", current_pos_x_left)
print("Current y pos is:", current_pos_y_left)

#Saving pos2
pos2 = [current_pos_x_left,current_pos_y_left]


print ("Saved right arm pos1 is:",pos1[0],pos1[1])
print ("Saved left arm pos2 is",pos2[0],pos2[1])

#Both arms goes back to INIT_posture
H.quick_send(
    topic="/trajectory",
    print_message="Front probing complete. Retracting both arms to INIT posture"
    wait_completion=True,
    postures=[
        INIT_posture,
])

#Moves the left arm to the left of the tray
H.quick_send(
    topic="/trajectory",
    print_message="Moving left arm to the side",
    wait_completion=True,
    postures=[
        PROBE_posture_side,
])

#Moves left arm in position to probe right in order to locate the tray
PROBE_posture_side2 = H.posture(
    7,
    (H.cm(35, -10, 17.3), H.e2q(180, 0, -90), 0.0),
    (H.cm(pos2[0]*100 + 10, 35, 15.3), H.e2q(180, 0, 0), 0.0)
)

H.quick_send(
    topic="/trajectory",
    print_message="Ready for probing position 3",
    wait_completion=True,
    postures=[
        PROBE_posture_side2,
])
```

```
#The left arm moves incrementally to the right until the tray triggers the force
while(current_force_x_left <0.2):
    H.quick_send(
        topic="/trajectory",
        print_message="Probing position 3 with left arm",
        wait_completion=True,
        postures=[
            PROBE_posture_side_inc,
])

#Saving pos2
pos3 = [current_pos_x_left,current_pos_y_left]
print ("Saved left arm pos3 is:",pos3[0],pos3[1])

#Left arm goes back to the position before probing at the side
H.quick_send(
    topic="/trajectory",
    print_message="Side probing complete",
    wait_completion=True,
    postures=[
        PROBE_posture_side2,
])

#Goes to ready pose
H.quick_send(
    topic="/trajectory",
    routine_name="ready_pose",
    print_message="READY_POSE sent")

#The positions for picking up the tray is calculated with calc_geometry
pickup_pose = calc_geometry(pos1[0], pos2[0],pos3[0], pos1[1], pos2[1], pos3[1])
print(pickup_pose)

#Moves to a position behind their pickup positions and open their grippers
pre_pickup_posture = H.posture(
    10,
    (H.cm(pickup_pose[2]*100-5,pickup_pose[3]*100, 16.8), H.e2q(0, 90, 0), 20.0)
    (H.cm(pickup_pose[0]*100-5,pickup_pose[1]*100, 16.8), H.e2q(0, 90, 0), 20.0)
)

#Moves forward to the pickup positions
pickup_posture2 = H.posture(
    5,
    primary=(H.cm(10, 0, 0), H.e2q(0, 0, 0), 0.0),
    secondary=(H.cm(10, 0, 0), H.e2q(0, 0, 0), 0.0),
    incremental=H.Incr.GLOBAL
```

```
)

#Grippers closes
close_posture = H.posture(
    2,
    (H.cm(pickup_pose[2]*100+5,pickup_pose[3]*100, 16.8), H.e2q(0, 90, 0), -10.
    (H.cm(pickup_pose[0]*100+5,pickup_pose[1]*100, 16.8), H.e2q(0, 90, 0), -10.0
)

#Raises the tray
lifting_posture = H.posture(
    4,
    primary=(H.cm(0, 0, 5), H.e2q(0, 0, 0), 0),
    secondary=(H.cm(0, 0, 5), H.e2q(0, 0, 0), 0),
    incremental=H.Incr.GLOBAL
)

#Lowers the tray to put it back
placing_posture = H.posture(
    5,
    primary=(H.cm(0, 0, -5), H.e2q(0, 0, 0), 0),
    secondary=(H.cm(0, 0, -5), H.e2q(0, 0, 0), 0),
    incremental=H.Incr.GLOBAL
)

#Grippers opens when the tray is placed
release_posture = H.posture(
    3,
    (H.cm(pickup_pose[2]*100+5,pickup_pose[3]*100, 16.8), H.e2q(0, 90, 0), 20.0)
    (H.cm(pickup_pose[0]*100+5,pickup_pose[1]*100, 16.8), H.e2q(0, 90, 0), 20.0)
)

release_posture2 = H.posture(
    3,
    primary=(H.cm(0, 0, 0), H.e2q(0, 0, 0), 20.0),
    secondary=(H.cm(0, 0, 0), H.e2q(0, 0, 0), 20.0),
    incremental=H.Incr.GLOBAL
)

#Arms retract from the tray
backing_posture = H.posture(
    3,
    primary=(H.cm(-10, 0, 0), H.e2q(0, 0, 0), 0.0),
    secondary=(H.cm(-10, 0, 0), H.e2q(0, 0, 0), 0.0),
    incremental=H.Incr.GLOBAL
)
```

```
#Sending trajectories to pick up the tray
H.quick_send(
    topic="/trajectory",
    print_message="Ready to grip",
    wait_completion=True,
    postures=[
        pre_pickup_posture,
        pickup_posture2,
        close_posture,
])

rospy.sleep(1)

H.quick_send(
    topic="/trajectory",
    print_message="Lifting the tray",
    wait_completion=True,
    postures=[
        lifting_posture,
])

#Sleipner moves 1 m backwards, then back 1 m forwards
move_sleipner(1, 0, 0, 8, sleipner_pub)
rospy.sleep(10)
move_sleipner(-1, 0, 0, 8, sleipner_pub)
rospy.sleep(10)

#Sending trajectories to place down the tray
H.quick_send(
    topic="/trajectory",
    print_message="Placing down and releasing the tray",
    wait_completion=True,
    postures=[
        placing_posture,
        release_posture,
        release_posture2,
        backing_posture,
])

#Returning to ready pose before the program is finished
H.quick_send(
    topic="/trajectory",
    routine_name="ready_pose",
    print_message="Routine complete. Returning to ready posture")


rospy.sleep(1)
```

```
if __name__ == "__main__":
    main()
```